

Interview NoSQL

What is the main purpose of NoSQL databases?

The main purpose of NoSQL databases is to provide a flexible and scalable way to store and manage data, especially for modern applications that require handling large volumes of diverse and rapidly changing data. Here are some key points:

- **Flexibility:** NoSQL databases allow for unstructured or semi-structured data, meaning you can store different types of data without a fixed schema.
- **Scalability:** They can easily scale horizontally, which means you can add more servers to handle increased data and traffic without significant changes to the database structure.
- **High Availability:** NoSQL databases are designed to be distributed, providing fault tolerance and ensuring that the system remains available even if some parts fail.
- **Performance:** They are optimized for high performance, allowing for quick read and write operations, which is essential for applications with a large number of users.

Overall, NoSQL databases are particularly useful for applications like social media platforms, e-commerce sites, and big data analytics, where traditional relational databases may struggle to keep up with the demands.

How can NoSQL databases be applied in real-world scenarios?

NoSQL databases can be applied in various real-world scenarios. Here are two detailed examples:

1. Social Media Platforms:

- **Use Case:** Managing user profiles, posts, comments, and relationships.
- **Application:** A social media platform can use a document database (like MongoDB) to store user profiles, which may include various fields such as name, age, interests, and profile pictures. Each user can have a different structure, allowing for flexibility. Additionally, a graph database (like Neo4j) can be used to manage relationships between users, such as friendships and followers, enabling efficient queries about connections and interactions.

2. E-commerce Websites:

- **Use Case:** Handling product catalogs, user reviews, and shopping carts.
- **Application:** An e-commerce site can utilize a key-value store (like Redis) to manage user sessions and shopping carts, allowing for quick access to frequently accessed data. A column-family database (like Cassandra) can be used to store product information, where each product can have different attributes (like size, color, and price) without needing a fixed schema. This flexibility helps the site adapt to changing inventory and user preferences.

These examples illustrate how NoSQL databases can effectively support the diverse and dynamic data needs of modern applications.

How would you implement NoSQL in a large-scale application?

Implementing NoSQL in a large-scale application involves several key steps. Here's a concise outline of the process:

1. Define Requirements:

- Identify the types of data you need to store (e.g., user profiles, transactions, logs).
- Determine the expected scale (number of users, data volume) and performance requirements (read/write speeds).

2. Choose the Right NoSQL Database:

- Select a NoSQL database type based on your needs:
 - **Document databases** (e.g., MongoDB) for flexible data structures.
 - **Key-value stores** (e.g., Redis) for fast access to data.
 - **Column-family stores** (e.g., Cassandra) for handling large datasets with high write and read throughput.
 - **Graph databases** (e.g., Neo4j) for managing complex relationships.

3. Design the Data Model:

- Create a schema-less design that allows for flexibility in data storage.
- Organize data in a way that optimizes for the most common queries and access patterns.

4. Set Up the Infrastructure:

- Deploy the NoSQL database on cloud services or on-premises servers, ensuring it can scale horizontally.
- Configure replication and sharding to distribute data across multiple nodes for fault tolerance and performance.

5. Implement Data Access Layer:

- Develop an API or data access layer that interacts with the NoSQL database, ensuring efficient data retrieval and manipulation.

6. Monitor and Optimize:

- Use monitoring tools to track performance, latency, and resource usage.
- Optimize queries and data structures based on usage patterns and performance metrics.

7. Test and Iterate:

- Conduct thorough testing to ensure the application meets performance and reliability standards.
- Iterate on the design and implementation based on feedback and changing requirements.

By following these steps, you can effectively implement NoSQL in a large-scale application, ensuring it meets the demands of modern data management.

How do noSql databases difer from relational databases?

NoSQL databases differ from relational databases in several key ways. Here's a concise comparison:

1. Data Structure:

- **NoSQL:** Uses flexible data models (document, key-value, column-family, graph) that allow for unstructured or semi-structured data. There is no fixed schema.
- **Relational:** Uses a structured format with fixed schemas, organizing data into tables with rows and columns.

2. Scalability:

- **NoSQL:** Designed for horizontal scaling, allowing you to add more servers easily to handle increased load and data.
- **Relational:** Typically scales vertically, meaning you need to upgrade existing hardware to improve performance.

3. Transactions and Consistency:

- **NoSQL:** Often follows the BASE (Basically Available, Soft state, Eventually consistent) model, allowing for more flexible consistency.
- **Relational:** Follows the ACID (Atomicity, Consistency, Isolation, Durability) model, ensuring strict consistency and reliability in transactions.

4. Query Language:

- **NoSQL:** Uses various query languages or APIs specific to the database type, which may not be standardized.

- **Relational:** Uses SQL (Structured Query Language), a standardized language for querying and managing data.

5. Use Cases:

- **NoSQL:** Best suited for applications with large volumes of diverse data, such as social media, big data analytics, and real-time web applications.
- **Relational:** Ideal for applications requiring complex queries and transactions, such as financial systems and traditional business applications.

These differences highlight how NoSQL databases provide flexibility and scalability for modern applications, while relational databases offer structured data management and strong consistency.

What is the main advantage of NoSQL databases over relational databases?

The main advantage of NoSQL databases over relational databases is **flexibility in data modeling**. Here are some key points that highlight this advantage:

- **Schema-less Design:** NoSQL databases allow for unstructured or semi-structured data, meaning you can store different types of data without a fixed schema. This flexibility makes it easier to adapt to changing data requirements.
- **Scalability:** NoSQL databases are designed to scale horizontally, allowing you to add more servers to handle increased data and traffic. This is particularly beneficial for applications that experience rapid growth.
- **High Performance:** They are optimized for high-speed read and write operations, making them suitable for applications that require real-time data access and processing.
- **Handling Diverse Data Types:** NoSQL databases can efficiently manage various data types, such as documents, key-value pairs, graphs, and wide-

column stores, which is essential for modern applications that deal with diverse data.

Overall, the flexibility and scalability of NoSQL databases make them particularly advantageous for applications that need to handle large volumes of rapidly changing data.

What would happen if relational databases were used for large-scale applications?

Using relational databases for large-scale applications can lead to several challenges:

1. Scalability Issues:

- **Vertical Scaling:** Relational databases typically require vertical scaling, meaning you need to upgrade existing hardware to handle increased load. This can become expensive and may reach a limit where further upgrades are not feasible.

2. Performance Bottlenecks:

- **Complex Queries:** As the volume of data grows, complex queries can become slow and resource-intensive, leading to performance bottlenecks.
- **Locking and Contention:** High levels of concurrent transactions can lead to locking issues, where multiple processes compete for access to the same data, causing delays.

3. Rigid Schema:

- **Schema Changes:** Modifying the database schema to accommodate new data types or structures can be complex and time-consuming, often requiring downtime or extensive migration efforts.

4. Limited Flexibility:

- **Data Variety:** Relational databases are not well-suited for handling unstructured or semi-structured data, which is increasingly common in

modern applications.

5. **Cost of Maintenance:**

- **Resource Intensive:** Maintaining a large relational database can require significant resources, including database administrators and specialized hardware.

In summary, while relational databases are effective for many applications, they may struggle to meet the demands of large-scale applications that require flexibility, scalability, and high performance.

What is the main difference between NoSQL and relational databases?

The main difference between NoSQL and relational databases lies in their data models and structure. Here's a concise comparison:

- **Data Model:**
 - **NoSQL:** Utilizes flexible data models (document, key-value, column-family, graph) that allow for unstructured or semi-structured data. There is no fixed schema, enabling easy adaptation to changing data requirements.
 - **Relational:** Employs a structured format with fixed schemas, organizing data into tables with rows and columns. Each table must adhere to a predefined schema.
- **Scalability:**
 - **NoSQL:** Designed for horizontal scaling, allowing the addition of more servers to handle increased data and traffic efficiently.
 - **Relational:** Typically scales vertically, requiring upgrades to existing hardware, which can be limiting and costly.
- **Consistency Model:**
 - **NoSQL:** Often follows the BASE (Basically Available, Soft state, Eventually consistent) model, allowing for more flexible consistency.

- **Relational:** Adheres to the ACID (Atomicity, Consistency, Isolation, Durability) model, ensuring strict consistency and reliability in transactions.

These differences highlight how NoSQL databases provide flexibility and scalability for modern applications, while relational databases offer structured data management and strong consistency.

What would happen if relational databases were used for large-scale applications?

Using relational databases for large-scale applications can lead to several challenges:

1. Scalability Limitations:

- **Vertical Scaling:** Relational databases typically require vertical scaling, meaning you need to upgrade existing hardware to handle increased load. This can become expensive and may reach a limit where further upgrades are not feasible.

2. Performance Bottlenecks:

- **Complex Queries:** As the volume of data grows, complex queries can become slow and resource-intensive, leading to performance bottlenecks.
- **Locking and Contention:** High levels of concurrent transactions can lead to locking issues, where multiple processes compete for access to the same data, causing delays.

3. Rigid Schema:

- **Schema Changes:** Modifying the database schema to accommodate new data types or structures can be complex and time-consuming, often requiring downtime or extensive migration efforts.

4. Limited Flexibility:

- **Data Variety:** Relational databases are not well-suited for handling unstructured or semi-structured data, which is increasingly common in modern applications.

5. **Cost of Maintenance:**

- **Resource Intensive:** Maintaining a large relational database can require significant resources, including database administrators and specialized hardware.

In summary, while relational databases are effective for many applications, they may struggle to meet the demands of large-scale applications that require flexibility, scalability, and high performance.

What scenarios would benefit from using NoSQL over relational databases?

NoSQL databases are particularly beneficial in the following scenarios:

1. **Big Data Applications:**

- When dealing with large volumes of data that are constantly growing, NoSQL databases can efficiently handle the scale and variety of data.

2. **Real-Time Analytics:**

- Applications that require real-time data processing and analytics, such as social media platforms or online gaming, benefit from NoSQL's high-speed read and write capabilities.

3. **Unstructured or Semi-Structured Data:**

- When the data does not fit neatly into tables (e.g., JSON documents, images, or logs), NoSQL databases can store and manage this data more effectively.

4. **Rapidly Changing Requirements:**

- In environments where data models frequently change, NoSQL's schema-less design allows for quick adaptations without extensive downtime or

migration efforts.

5. **Distributed Systems:**

- Applications that require high availability and fault tolerance, such as e-commerce platforms or content delivery networks, can leverage NoSQL's distributed architecture.

6. **High Traffic Applications:**

- Websites and applications that experience high traffic and need to scale horizontally to accommodate user demand can benefit from NoSQL's ability to distribute data across multiple servers.

7. **Social Networks:**

- Social media platforms often require complex relationships and connections between users, which can be efficiently managed using graph databases, a type of NoSQL database.

These scenarios highlight how NoSQL databases provide the flexibility, scalability, and performance needed for modern applications.

What are the advantages of using NoSQL databases in application?

Using NoSQL databases in applications offers several advantages:

1. **Scalability:**

- **Horizontal Scaling:** NoSQL databases can easily scale out by adding more servers, allowing them to handle increased data loads and user traffic efficiently.

2. **Flexibility:**

- **Schema-less Design:** They allow for unstructured or semi-structured data, enabling developers to adapt to changing data requirements without the need for complex schema migrations.

3. High Performance:

- **Fast Read and Write Operations:** NoSQL databases are optimized for high-speed data access, making them suitable for applications that require real-time processing.

4. Variety of Data Models:

- **Diverse Storage Options:** NoSQL databases support various data models (document, key-value, column-family, graph), allowing developers to choose the best fit for their specific use case.

5. Handling Big Data:

- **Efficient Data Management:** They are designed to manage large volumes of data, making them ideal for big data applications and analytics.

6. High Availability and Fault Tolerance:

- **Distributed Architecture:** Many NoSQL databases are built to be distributed, providing native support for high availability and fault tolerance, which is crucial for mission-critical applications.

7. Cost-Effectiveness:

- **Lower Infrastructure Costs:** By using commodity hardware and cloud-based solutions, organizations can reduce costs associated with maintaining high-performance databases.

These advantages make NoSQL databases a strong choice for modern applications that require flexibility, scalability, and high performance.

How can NoSQL databases improve application performance in real scenarios?

NoSQL databases can significantly improve application performance in real scenarios through several mechanisms:

1. Faster Data Access:

- **Optimized for Speed:** NoSQL databases are designed for high-speed read and write operations. For example, a social media application can quickly retrieve user profiles and posts, enhancing user experience.

2. Horizontal Scalability:

- **Load Distribution:** By distributing data across multiple servers, NoSQL databases can handle increased traffic without performance degradation. For instance, an e-commerce platform can manage high traffic during sales events by adding more nodes to the database cluster.

3. Flexible Data Models:

- **Adaptability:** Applications that require frequent changes to data structures can benefit from NoSQL's schema-less design. For example, a content management system can easily accommodate new content types without extensive database redesign.

4. Efficient Handling of Unstructured Data:

- **Diverse Data Types:** NoSQL databases can store and manage various data formats (e.g., JSON, images, logs) efficiently. A logging application can quickly ingest and analyze log data in real-time, improving monitoring and troubleshooting.

5. High Availability:

- **Fault Tolerance:** Many NoSQL databases are built to be distributed, ensuring that data remains accessible even in the event of server failures. For example, a financial application can maintain uptime and reliability during peak transaction periods.

6. Real-Time Analytics:

- **Immediate Insights:** NoSQL databases can support real-time analytics, allowing applications to provide instant feedback and insights. For instance, a recommendation engine can analyze user behavior in real-time to suggest products immediately.

These improvements in performance can lead to enhanced user experiences, increased efficiency, and better overall application responsiveness.

1. What are the four main categories of NoSQL databases?

- ☐ Key-value, document, common, graph
- ☐ Key-value, relational, column, graph
- ☒ Key-value, document, column, graph

Correct! The four major categories of NoSQL databases all have unique characteristics that make them great fits for diverse types of applications or parts of applications.

- ☐ Key-value, distributed, column, graph

2. What is the name of the fully managed service model that many NoSQL databases now leverage?

- ☐ Platform as a service (PaaS)
- ☒ Database as a service (DBaaS)

Correct! Database as a service (DBaaS) is the fully managed service model that many NoSQL databases now leverage.

- ☐ Software as a service (SaaS)
- ☐ Network as a service (NaaS)

3. What is the most common trait among all NoSQL databases?

- ☐ All NoSQL databases are open source.
- ☐ All NoSQL databases are relational.
- ☐ All NoSQL databases are document-based.
- ☒ All NoSQL databases are non-relational.

Correct! The most common trait among all NoSQL databases is that they are non-relational in nature.

4. Which document formats are typically used for documents stored in a document-based NoSQL database?

- ☐ XML and CSV format
- ☐ PDF and JSON format
- ☒ XML and JSON format

Correct! Document-based NoSQL databases typically store documents in XML or JSON format.

- ☐ CSV and JSON format

1. What common trait do the NoSQL family of databases share?

- ☐ Technology
- ☒ Non-relational
- ☐ Tabular style
- ☐ Exclusion of SQL

✓ **Correct**

Correct! This is a family of databases that vary widely in style and technology, but which all share a common trait in that they are non-relational in nature (they are not a standard row and column relational database management system).

2. What might be the most common reason to use a NoSQL database?

- ☒ Scalability
- ☐ Normalization
- ☐ Security
- ☐ Joins

✓ **Correct**

Correct! The elasticity of scaling both up and down to meet the varying demands of applications is key.

Question 1

What common trait do the NoSQL family of databases share?

✓ **Non-relational**

(NoSQL databases are primarily characterized by their non-relational, flexible data models, as opposed to traditional relational databases.)

Question 2

What might be the most common reason to use a NoSQL database?

✓ **Scalability**

(NoSQL databases excel at horizontal scalability, making them ideal for handling large volumes of data and high-throughput applications.)

3. What makes key-value NoSQL databases powerful for read operations?

- ☐ They are good for complex join operations.
- ☒ They are represented as a hashmap.
- ☐ They are good for bulk reads.
- ☐ They shard easily across nodes.

✓ **Correct**

Correct! Because key-value stores are represented as a hashmap, they are powerful for read operations.

Question 3

What makes key-value NoSQL databases powerful for read operations?

✓ **They are represented as a hashmap.**

(Key-value stores use a simple hashmap structure, enabling extremely fast read/write operations by direct key access.)

How can you decide when to use RDBMS or NoSQL?

Deciding between RDBMS and NoSQL depends on your specific needs. Here are some key points to consider:

Use RDBMS when:

- **Data is structured:** You have a clear schema with defined relationships (like tables).
- **Consistency is crucial:** You need strong data integrity and support for multi-document transactions.
- **Complex queries are required:** You need to perform complicated joins and queries across multiple tables.
- **You have a moderate amount of data:** RDBMS can handle a reasonable amount of data efficiently.

Use NoSQL when:

- **Data is unstructured or semi-structured:** You need flexibility in how data is stored (like JSON documents).
- **Scalability is important:** You expect to handle large volumes of data and need to scale out easily.
- **High availability is needed:** You want a system that remains operational even during failures.
- **Performance is a priority:** You need fast read and write operations, especially for large datasets.

By analyzing your project requirements against these points, you can make a more informed decision.

What is the main difference between RDBMS and NoSQL?

The main differences between RDBMS and NoSQL can be summarized as follows:

1. Data Structure:

- **RDBMS:** Uses a structured format with tables, rows, and columns. Data is organized in a predefined schema.
- **NoSQL:** Supports various data formats (document, key-value, column, graph) and is more flexible with schema, allowing unstructured or semi-structured data.

2. Consistency:

- **RDBMS:** Follows ACID (Atomicity, Consistency, Isolation, Durability) properties, ensuring strong consistency and data integrity.
- **NoSQL:** Often follows BASE (Basically Available, Soft state, Eventually consistent) principles, which prioritize availability and scalability over immediate consistency.

3. Scalability:

- **RDBMS:** Typically scales vertically (adding more power to a single server), which can be limiting.
- **NoSQL:** Designed to scale horizontally (adding more servers), making it easier to handle large volumes of data.

4. Query Language:

- **RDBMS:** Uses SQL (Structured Query Language) for querying data.
- **NoSQL:** Uses various query languages or APIs depending on the database type (e.g., MongoDB uses its own query language).

These differences help determine which type of database is best suited for specific applications and use cases.

How you can migrating from RDBMS to Nosql database?

Migrating from RDBMS to a NoSQL database involves several key steps. Here's a simplified process:

1. Assess Requirements:

- Identify the reasons for migration (e.g., scalability, flexibility).
- Analyze the data structure and usage patterns in your current RDBMS.

2. Choose the Right NoSQL Database:

- Select a NoSQL database type (document, key-value, column, graph) based on your data needs and application requirements.

3. Redesign the Data Model:

- In NoSQL, the data model is driven by how the application queries the data, not just by the data itself.
- Denormalize your data: Unlike RDBMS, where data is normalized, NoSQL often requires storing related data together to optimize for read performance.

4. Migrate Data:

- Extract data from the RDBMS and transform it into the NoSQL format.
- Load the transformed data into the NoSQL database using appropriate tools or scripts.

5. Update Application Logic:

- Modify your application code to interact with the NoSQL database, using the new query language or API.

6. Test Thoroughly:

- Conduct extensive testing to ensure that the application works correctly with the new database.
- Validate data integrity and performance.

7. Monitor and Optimize:

- After migration, monitor the system for performance and make adjustments as needed.

This process requires careful planning and execution to ensure a smooth transition.

What challenges might arise during the migration process?

Migrating from RDBMS to NoSQL can present several challenges, including:

1. Data Model Redesign:

- **Challenge:** RDBMS uses a structured schema, while NoSQL often requires a denormalized and flexible schema. Adapting to this new model can be complex.
- **Solution:** Carefully analyze how your application queries data and redesign the data model accordingly.

2. Data Integrity and Consistency:

- **Challenge:** NoSQL databases may not enforce strict consistency like RDBMS, leading to potential data integrity issues.
- **Solution:** Implement application-level checks and balances to maintain data integrity.

3. Query Language Differences:

- **Challenge:** Transitioning from SQL to NoSQL query languages can be difficult for developers familiar with SQL.
- **Solution:** Provide training and resources to help the team learn the new query language and paradigms.

4. Performance Tuning:

- **Challenge:** NoSQL databases may require different performance tuning strategies compared to RDBMS.
- **Solution:** Monitor performance closely after migration and adjust configurations as needed.

5. Tooling and Ecosystem:

- **Challenge:** Existing tools and processes may not be compatible with NoSQL databases.
- **Solution:** Research and adopt new tools that work well with the chosen NoSQL database.

6. Data Migration Complexity:

- **Challenge:** Extracting, transforming, and loading (ETL) data can be complex, especially with large datasets.
- **Solution:** Plan the migration process carefully, possibly using automated ETL tools to streamline the process.

7. Team Skill Gaps:

- **Challenge:** Your team may lack experience with NoSQL technologies.
- **Solution:** Invest in training and resources to build the necessary skills within the team.

By anticipating these challenges and planning accordingly, you can facilitate a smoother migration process.

How can you ensure data integrity during migration?

Ensuring data integrity during migration from RDBMS to NoSQL involves several strategies:

1. Data Validation:

- **Before Migration:** Validate the data in the RDBMS to ensure it is clean and consistent. Check for duplicates, missing values, and data type mismatches.
- **After Migration:** Perform checks to confirm that all data has been accurately transferred and matches the original data.

2. Use of Transactional Processes:

- Implement transactional processes during the migration to ensure that data is only committed to the NoSQL database if all parts of the migration are successful. This helps prevent partial data transfers.

3. Implement Application-Level Constraints:

- Since NoSQL databases may not enforce strict constraints, implement application-level checks to ensure that data adheres to business rules and integrity constraints.

4. Incremental Migration:

- Consider migrating data in smaller batches rather than all at once. This allows for easier monitoring and validation of data integrity at each step.

5. Backup and Rollback Plans:

- Create backups of the original RDBMS data before starting the migration. Have a rollback plan in place to restore data if issues arise during the migration process.

6. Testing and Verification:

- Conduct thorough testing after migration to verify that the data is intact and functional. This includes running queries to check for expected results and comparing counts of records.

7. Monitor Data Consistency:

- After migration, continuously monitor the NoSQL database for data consistency and integrity issues. Implement logging to track any anomalies.

By following these strategies, you can help ensure that data integrity is maintained throughout the migration process.

1. What is one way that a distributed NoSQL database usually shards data?

- ☐ By distributing all records that share the same key across multiple servers
- ☐ By grouping all records that have the same data on the same server
- ☒ By grouping all keys lexically

Correct! All distributed systems need to expose a way to store a large chunk of data. For example, all keys that start with the letter “A” can be found on a specific server.

- ☐ By grouping all keys numerically

2. Which requirement would prompt you to consider choosing RDBMS over NoSQL?

- ☐ Flexible schema
- ☐ High availability
- ☐ Easy scalability
- ☒ Complicated joins

Correct! NoSQL systems, by design, do not support transactions and joins (except in limited cases).

3. What requirement does the BASE model support?

- ☐ Isolated consistency
- ☐ Atomic consistency
- ☐ Immediate consistency
- ☒ Eventual consistency

Correct! A BASE data store values availability over consistency, but it doesn't offer guaranteed consistency of replicated data at write time.

4. What is the best way to describe the partition tolerance characteristic of the CAP theorem?

- ☐ Provides guarantees for both strong consistency and high availability
- ☒ The system continues to operate despite data loss or network failures.

Correct! Partition tolerance means that a given system continues operating even under data loss or network failure circumstances. The cluster must continue to work despite any number of communication breakdowns between nodes in the system.

- ☐ The database is highly available.
- ☐ Ensures data consistency even when a partition occurs

1. Which industry will almost exclusively use ACID databases?

- ☐ Online social networks
- ☐ E-commerce
- ☐ Marketing consultants
- ☒ Financial institutions

✔ **Correct**

Correct! Financial institutions will almost exclusively use ACID databases for money transfers because these operations depend on the atomic nature of ACID transactions.

2. What is data sharding?

- ☐ Retrieval of data from a failed node
- ☒ Partitioning the data
- ☐ Deletion of all data and schema
- ☐ Replication of data across multiple nodes

✔ **Correct**

Correct! This process breaks data into partitions for storage in a distributed system.

Question 1

Which industry will almost exclusively use ACID databases?

✔ **Financial institutions**

(Banks and financial systems require strict ACID compliance for transactions to ensure data integrity, consistency, and reliability.)

Question 2

What is data sharding?

✔ **Partitioning the data**

(Sharding splits a database into smaller, more manageable pieces (shards) distributed across different servers to improve scalability and performance.)

3. How many characteristics of the CAP theorem can a distributed system guarantee?

- ☐ None
- ☐ One
- ☒ Two
- ☐ Three

✔ **Correct**

Correct! A distributed system can guarantee delivery of only two of the three characteristics necessary for the successful design, implementation, and deployment of applications.

4. What drives the design of a data model in NoSQL?

- ☒ Your queries and the way the app accesses the data
- ☐ How the data is denormalized
- ☐ How the data is stored in one or more tables
- ☐ The choice between availability and consistency

✔ **Correct**

Correct! In NoSQL, the way your application accesses the data for the queries you are going to make is the driver. Models should be based on how the app interacts with the data rather than how the model can be stored as rows in one or more tables.

Question 3

How many characteristics of the CAP theorem can a distributed system guarantee?

☒ Two

*(The CAP theorem states that a distributed system can only guarantee **two** out of **Consistency, Availability, and Partition Tolerance** at the same time.)*

Question 4

What drives the design of a data model in NoSQL?

☒ Your queries and the way the app accesses the data

(NoSQL databases are optimized for specific query patterns, so the data model should align with how the application retrieves and processes data.)